

Predicting Future Digital Democracy Tipsheet Newsworthiness Scores with Machine Learning

Kavya Karunakaran

Advisor: Foaad Khosmood

1 Introduction

This project explores adding a machine learning component to the Tipsheets feature of the Digital Democracy Project. Digital Democracy is a website that provides information about legislative processes in California in order to make that information more accessible to journalists. The website has a feature called Tipsheets, which are AI-generated summaries of legislative hearings that highlight the most valuable information from a hearing. The goal of tipsheets is to help journalists easily find hearings to write about without having to listen to hearings in full. Each tipsheet has an associated score that indicates how “newsworthy” the hearing associated with the tipsheet is. The newsworthiness score indicates how likely a story is to be written about or covered in media, with a high score indicating high likelihood of being written about. Currently, the newsworthiness score can only be generated after the tipsheet has already been written about. With this project, we used machine learning algorithms to predict future tipsheet scores based on the previous tipsheet scores. We used multiple machine learning algorithms trained on a pool of past tipsheets with different features of the tipsheets, and were able to predict future tipsheet scores with an accuracy of 82.08%.

2 Background and Tools

First, a user must understand some background about the data being used. Tipsheets are explained in the Introduction section above. Tipsheets are based on legislative hearings—When a bill is proposed, it goes through multiple rounds of hearings with legislative committees, during which the bill is discussed and changes are made. This happens before the bill can be voted on. Depending on the bill, these hearings can have extremely important information, and tipsheets allow for journalists to quickly see summaries of the hearings without having to go through hundreds of hours of audio and transcripts. The newsworthiness scores aim to aid reporters further in identifying at a glance which hearings might be the most important to write about.

In this paper, we build a model for supervised machine learning. In supervised machine learning, we use features to predict a label, and the correct labels are known in training. In this case, the features are the text of the hearing transcript or the text of a bill analysis document. The hearing transcript is a transcript of everything that was said in the hearing. The bill analysis document is a document published by the committee that held the hearing and it is the committee’s summary of what was discussed and who was present. We use processed and unprocessed versions of each of those texts as features. The labels are the tipsheet’s newsworthiness score.

To process the text before using it, we apply stemming and stopwords removal. Both are natural language processing strategies, and we used the NLTK library to apply them. Stemming removes prefixes and suffixes to reduce words down to their base. This allows for words that are different conjugations or versions of the same word to be treated the same. Stopword removal is removing common, less important filler words like “the”, “a” and “of” from the text.

To evaluate the results of running our models, we calculated accuracy scores, applied cross validation, and created confusion matrices. We first split the data into testing and training data, then ran cross validation on the training data. Cross validation involves splitting the data into k folds, and running each combination of using k-1 folds as training data and the last fold as testing data. Then, we calculate accuracy by calculating the number of correct predictions out of total predictions made for each combination, and taking the average of the accuracy scores. We also calculated the accuracy after making predictions on the testing data. Finally, a confusion matrix is a table that compares the predicted labels to the actual labels so that we can visually see which labels and how many data points of each label are being misclassified.

We also occasionally applied balancing, which is a technique for dealing with imbalanced data where there are uneven numbers of data points corresponding to each label. We specifically used subsampling to do this, which means taking a smaller sample from each label so that the number of points for each label is equal.

We worked in Python. In addition to NLTK, we also employed Scikit-Learn, which is a machine learning library that provides machine learning models and methods for cross validation and generating confusion matrices. We also used Pandas to view and manipulate the data.

We tried several machine learning models. Naive Bayes is a natural language processing model commonly used on text. It calculates the probability of each label, given a set of features. We then used Random Forest, which employs multiple decision trees and then calculates an average of the trees. Next we tried Support Vector Machines, which aims to find the best hyperplane that separates the data into classes. Finally, we used Boosting, which uses multiple models sequentially, and each model works on fixing the errors of the previous models.

3 Related Work

1. Using machine learning of previous known material to predict newsworthiness of future events

The following papers discuss similar projects exploring using previous known information to predict the newsworthiness of events in the future.

In one paper (Trilling), the authors develop a theory of “shareworthiness”, related to newsworthiness, to predict how likely an news article is to be shared on social media. They identified several factors that increase an article’s likelihood of being shared, including the presence of domestic issues, a positive tone, and not being written by a news agency, among others. This idea is similar to the goal of our project, in that they are both attempting to predict how “interesting” or relevant a news story or event will be.

In another paper (Nishal), the authors trained a model on crowdsourced newsworthiness ratings to predict new scientific news articles' newsworthiness. They tried logistic regression, complement naive bayes, random forest, and extra trees. They achieved the most accurate results from extra trees, with a precision of 0.8 at 10 items and a precision of 0.67 at 15 items. While we discovered this paper after conducting our research, this paper provided helpful confirmation that our methods and the models we tried were similar to those tried in a very similar project.

In (Zhang), researchers trained machine learning models on health science models to identify factors of articles that have increased press and media coverage. They also achieved a high degree of success in identifying factors.

The researchers of another paper (Xiong) predicted the number of online clicks on a news article based on two new features that they developed—Attractiveness, or the appeal or interest of an article to the audience, and timeliness, or the recency of the news. They used deep learning rather than machine learning to develop their model.

In (Rybinski), the authors identified six basic emotions and explored how the emotions expressed in the news articles increased or decreased the stories' longevity. They found that articles with positive emotions expressed tend to have more longevity than primarily negative articles. They used LDA topic modeling, a form of unsupervised machine learning.

These papers were helpful to explore because they provided us with many examples of research with very similar objectives to our project being conducted. There are many papers in which researchers attempt to train models to predict the newsworthiness of articles in different areas or using different aspects of the articles or text. The papers provide us with ideas for next steps and types of features or models that have had success in the past with predicting newsworthiness.

2. Similar work in Digital Democracy

The following papers discuss other work that has been done as part of the Digital Democracy project using California legislative data.

The first paper (Latner) provides an introduction to the data covered in Digital Democracy and the aims of the project.

In another (Kauffman), researchers explored using bill discussion transcripts to predict legislators' alignments (as a score) with different organizations. They employed natural language processing machine learning methods to accomplish this, and achieved an accuracy of 78%.

The next paper (Klimashevskaya) covers the methods used to develop the AI model that generates the tipsheets. They achieved a high degree of accuracy in capturing the most important facts from a hearing in the tipsheets. They generated the tipsheets by identifying certain phenoms, or features of a hearing text that might be interesting points to be highlighted in a summary.

Another paper (Howe) explores features that reporters desire for generating tipsheets. They expand on the phenoms by adding new phenoms based on features that reporters would like to see highlighted in hearing summaries.

It was helpful to explore other work for the Digital Democracy project because there were other AI and machine learning models generated on the same set of data. While the models had different purposes than the one developed for this paper, these explorations helped identify strategies and models that work well with this data in particular. This data is unique from the other newsworthiness papers that we explored because it deals with legislative topics and politics. Exploring digital democracy helped acquaint ourselves to the quirks of the legislative data.

4 System and Development

We trained the models on past tipsheets in JSON form. We worked with a pool of all of the tipsheets from Jan 2023 to August 2024. We decided to use the tipsheets from Jan-Dec 2023 as training and Jan-Aug 2024 as testing data. This allowed us to have a larger pool of training data than testing data.

Before training models, the data needed to be cleaned and converted to a format that the models could use. First, we extracted the necessary columns from the JSON files and wrote the data to a TSV file. We did this because TSV files are easier to import and faster to read than hundreds of JSON files. We chose to use two main features of the JSON files as features in the models. The first was the transcripts of the hearings that the tipsheets were based on. The second was the text of bill analysis documents, which are documents published by committees as summaries of what happens in a hearing. Our reasoning for choosing these two features was that they both contain the contents of what was discussed in the hearing. The contents of the hearing is likely a strong predictor of how newsworthy its tipsheet will be. For each transcript or bill analysis, we created an array of all the words of the document and stored the array in the TSV. We added stemmed versions of each document. For the stemmed versions, along with the stemming, we removed stopwords and converted the words to lowercase.

We began creating models with a simple Naive Bayes model. First, we used Pandas to read the TSV and import it into a dataframe. We used the NLTK library's Naive Bayes model as a starting point. First, we separated the data into testing and training data. We did this by creating two new data frames by filtering the original data frame by the date, so that all data points before Dec 2023 were in the training data frame and all points after Dec 2023 were in the testing data frame. We used the transcript or bill analysis words as the feature and the tipsheet score as the label. Since Naive Bayes requires categorical variables as labels, we binned the scores into 0, 1-5, 6-14, and 15+. We chose to put 0 in a separate category from 1-5 because there was a high number of tipsheets with 0 scores. For the features, we created a dictionary where the keys were the words of the bill analysis or hearing transcript, and the values were the frequency of the keys. Naive Bayes required that the frequency also be categorical, so we created Low, Medium, and High bins. We generated bin boundaries for each word by calculating the frequency of each word across all the tipsheets, and then categorized the words of each tipsheet into the bins based on their frequency in that tipsheet. The resulting input to the Naive Bayes model was a list of tuples of the feature dictionary and label. For every model we trained, we ran the model first on the transcript words as features, then on the stemmed and stopword-removed transcript words, then

bill analysis text, and then the stemmed and stopword-removed bill analysis text. For each feature that we trained the model on, we displayed a confusion matrix that we computed using the training data, and we displayed the accuracy which we computed using the predictions on the test data. The confusion matrix revealed that there was a high rate of misclassified data points, and we theorized that this may be because the distribution of data points between the label bins was uneven. To combat this, we balanced the data by applying subsampling. The resulting accuracy was the highest for the unprocessed transcript words, at 36.73%. With the other three features, the accuracy stayed within a similar range, ranging from 30 to 35% accuracy.

To attempt to improve the model and get a higher accuracy, we created another Naive Bayes model with Scikit-Learn. We theorized that using the Multinomial Naive Bayes model of Scikit-Learn may lead to more accurate results. Scikit-Learn requires the data to be vectorized which NLTK does not, which may lead to better results. Additionally, the NLTK Naive Bayes model is a different version of Naive Bayes than Multinomial Naive Bayes. We created a pipeline of methods to run, train, and test Scikit-Learn machine learning models so that after running Naive Bayes, we could easily test other machine learning algorithms. In order to run the model, first we reformatted the data so that it would be in the format required by Scikit-Learn. Scikit-Learn required us to first create a vocabulary for the feature data. To do this, we created a Python set combining the feature words for every row. Using a Python set allowed us to eliminate duplicates in the list, so that we ended up with a list of every word that was present in the dataset. Then, Scikit-Learn models take in a feature matrix for X and y. The matrix is a list of arrays, where each array corresponds to one row of data. Each array is encoded with numbers representing the value for each word in the vocabulary. We mapped the ["low", "medium", "high"] encoding of word bins to [0, 1, 2] numerical encodings. If the vocab was ["dog", "cat", "fish"], then the feature vector for a row might look something like [2, 1, 0] depending on the bin for each word. 0 was the default value to represent words from the vocabulary that were not present in the row of data. Similarly, we created a numerical encoding for the label bins. We created a pipeline of methods that we could feed in the dataframe through and get the resulting feature matrix for X_train, y_train, X_test, and y_test. Having this pipeline set up would also make it easy to try other models supported by Scikit-Learn after trying Naive Bayes. For each model, after getting the training and testing feature matrices, we initialized and fit the Scikit-Learn model on the training data, and then we used the Scikit-Learn `cross_val_score()` method to run cross validation on the training data. We ran the method with 5 folds, and outputted the mean resulting score to observe the results. Finally, we used the trained model to make predictions on the test data, and then we used the Scikit-Learn `ConfusionMatrixDisplay.from_predictions()` to display a confusion matrix using the correct test values and the predictions.

One challenge we encountered in this process was working with the compute power restrictions of the environment we used. Prior to trying Scikit-Learn models, we had been working on Google Colab due to its ease for viewing data and results and because the notebook allowed us to have the output of runs saved to come back to view. However, when we began to

train the scikit models on the Google Colab notebook, the runtime environment would frequently exceed the allotted compute power and time out, disconnecting the runtime environment and forcing us to restart the run. After attempting the run on Kaggle notebooks and Python notebooks in VS code, we were able to run the code to completion as a normal Python file in VS Code. The runtime was also sped up by running on the Cal Poly 127x01 - 127x04 servers. We adapted the process to convert the dataframe to prepare data in the format needed by Scikit-Learn, train the model, compute cross validation, and generate predictions on test data in the VS code Python file in the Cal Poly server. Then we wrote the model, predictions, cross validation result into a pickle file, imported the pickle into the Google Colab notebook, and displayed the cross validation mean score and confusion matrix in the notebook. This process allowed us to utilize the compute power of the Cal Poly servers to effectively train and evaluate the model, but we could also save the results to view later in the Google Colab notebook.

We decided to use Multinomial Naive Bayes. When we trained the model with Multinomial Naive Bayes with scikit-learn, we got significantly improved results—The best results were with the unprocessed transcript words, at an accuracy of 78.36%. The other features also yielded accuracy scores between 76 and 77%. We chose to try Multinomial Naive Bayes because it is the model of Naive Bayes supported by Scikit-Learn that is most well-suited to working with text. We then tried balancing the data with subsampling as we did with the NLTK Naive Bayes model. However, this caused the results to have a significantly worse accuracy (42.31% for unprocessed transcript words), so we concluded that subsampling would not improve results and did not continue to use it.

Next, we tried Random Forest. With Random Forest, there are optional hyperparameters, so we started by tuning the parameters. To tune the parameters, we used Scikit-Learn's Grid Search method in order to easily specify values to try for each parameter and see which combination yielded the best result. We chose to focus on three parameters—`n_estimators`, `max_depth`, and `min_samples_leaf`. `N_estimators` specifies the number of trees, `max_depth` is the maximum depth of each tree, and `min_samples_leaf` is the minimum number of samples required in each leaf node. We chose to focus on these parameters because they are good starting points for text features. Tuning these parameters allows the model to recognize patterns without overfitting. For `n_estimators` we tried 50 and 100, for `max_depth` we tried 2, 4, 6, and None where None allows each tree to grow fully, and for `min_samples_leaf` we tried 1, 3, and 5. We chose these values because they represented the range of values that could be used for these parameters, so that we could try the full range to find the best values. We found that the best parameters for unprocessed transcript words were None for max depth, 1 for min_samples_leaf, and `n_estimators=50`. For the other three features, max depth and `n_estimators` were the same, and `min_samples_leaf` was 3. We then ran the models and evaluated the accuracy and confusion matrix through the same process as Naive Bayes, and got an accuracy of 82.08% for the unprocessed transcript words. The other features yielded accuracy scores between 79 and 81%.

Next, we ran Support Vector Machine and Boosting through Scikit-Learn, again with the same process of evaluation and displaying results. We attempted to tune three parameters for

Support Vector Machine. The first was C, the regularization parameter, where a low C value leads to more regularization, and a high C value results in trying to classify all points correctly (with the risk of overfitting). We tried values 0.1, 1, and 10. Next, we wanted to tune kernel, which specifies the type of decision boundary—We specified linear and rbf (Radial Basis Function). The last parameter was gamma, which specifies the curve flexibility (which only applies to rbf), where scale bases curve flexibility on data size and auto bases it on variance. We again attempted to use Grid Search to tune the parameters through the Python script in VS code. However, the process for Grid Search took an extremely long time to run in the Cal Poly servers—Even after running overnight, the process continued to run. We faced similar issues with time outs in Kaggle and Google Colab notebooks, and due to the time constraints of this project, and the fact that we were getting over 75% accuracy with the model without setting any parameters, we chose not to tune hyper parameters. We got the best results for the stemmed transcript words, with an accuracy of 82.07%. The other accuracy scores were between 79 and 82%.

Finally, we tried boosting. We chose to use the Gradient Boosting Classifier in Scikit because it is a common starting point for boosting algorithms and it can work well with medium-sized data sets and text features. Due to the aforementioned timing issues with tuning parameters, we did not tune parameters for boosting. The highest accuracy yielded was 81.99% for unprocessed transcript words, and the other features yielded accuracy scores between 79 and 81% accuracy.

5 Results and Evaluation

In order to use one of the models, the data must first be imported into a Pandas dataframe. Next, we call a function to vectorize the data and get the data into correct form to use Scikit-Learn. Then, we use Scikit-Learn to initialize and fit the model with the training data. The screenshot below shows the function we used to convert the data to the correct form for Scikit-Learn and split the data into testing and training data.

```
def train_predict_analyze_sklearn(df, column, freq_encoding, label_encoding, default_value):
    new_df = df.dropna(subset=[column])

    bin_dict = generate_bin_boundaries(new_df, column)

    # get vocabulary
    print("in train_predict_analyze_sklearn")
    all_data = new_df.apply(lambda row: row_to_tuple_2(row, bin_dict), axis=1).tolist()
    vocab = sorted(set(word for sample, _ in all_data for word in sample))

    X_train, y_train, X_test, y_test = split_train_test_SK(new_df, bin_dict, vocab, column, freq_encoding, label_encoding, default_value)
    return X_train, y_train, X_test, y_test
```

Next, we use a Scikit-Learn method to run cross validation and print the mean accuracy score. Finally, we use the test data to make predictions and display a confusion matrix. Next, the user must create a feature vector for the data for which they want to predict scores for. After calling model.predict(), the results can be written into a result vector that can then be displayed or exported.

The best model was Random Forest on the unprocessed transcript words, with an accuracy of 82.09%. Support Vector Machine, for both processed and unprocessed transcript words, was close behind with almost as high accuracy scores. Additionally, we observed that the best feature was transcript words without stemming and stopword removal because for every model we trained, it was the feature that yielded the highest accuracy score.

6 Conclusion

We learned that it is possible to predict a future tipsheet score with a high degree of accuracy, as shown in the high accuracy scores above. The best model was Random Forest on unprocessed transcript words. There were a few shortcomings of the project. Most notably, the time constraints on the project prevented us from exploring more models beyond the models we tried. We also found through the confusion matrices that despite the high accuracy, there was a high number of misclassified samples (See appendix). Even despite subsampling and balancing, we were not able to find a solution to this issue. We also were not able to tune the hyperparameters as much as possible. However, we learned a lot from this project about the efficacy of using machine learning models to predict the newsworthiness of a tipsheet from multiple different features, and we have multiple different models that predict with a high degree of accuracy. There is much to be explored here in further research.

7 Future Work

There are several steps we could take to explore this project further. Since we weren't able to tune the hyperparameters for Support Vector Machine and Boosting, we would tune the hyperparameters to try to achieve higher accuracy. The next step after this would be reorganizing the data so that instead of using the entire pool of tipsheets before a certain time as the training data to predict future tipsheet scores, we would group all tipsheets corresponding to a single bill together in chronological order. This way, we could follow the progression of a bill through hearings over time and use that progression to more accurately predict a future tipsheet score for the same bill. We could also try different features. For instance, we could train on the text of the bill that each tipsheet's hearing corresponds to. This would involve using the Digital Democracy database to acquire the bill texts. We could also train models that use a combination of features, such as bill analysis text and transcript words, or other combinations. Finally, we would also explore other machine learning models or deep learning models. Training a neural network would be worth exploring to see if it could yield better results than the machine learning models.

8 Bibliography

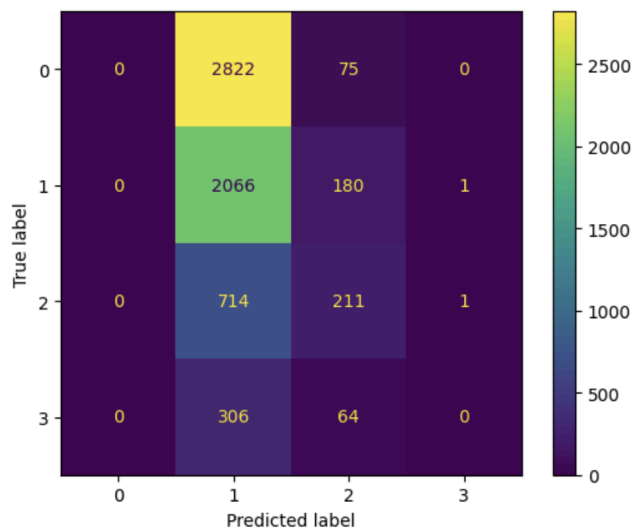
1. "AI4Reporters." *AI Tipsheets by CalMatters*, CalMatters, ai4reporters.org/login. Accessed 31 May 2025.
2. Howe, Patrick, et al. "Exploring Reporter-Desired Features for an AIGenerated Legislative News Tip Sheet." *International Symposium on Online Journalism*, International Symposium on Online Journalism, Apr. 2022, isoj.org/wp-content/uploads/2022/03/ISOJ-2022.pdf#page=17.

3. Kauffman, Daniel, et al. "Learning Alignments from Legislative Discourse." *ACM Digital Library*, Association for Computing Machinery, 30 May 2018, dl.acm.org/doi/abs/10.1145/3209281.3209413.
4. Klimashevskaiia, Anastasiia, et al. "Automatic News Article Generation from Legislative Proceedings: A Phenom-Based Approach." *ACM Digital Library*, Association for Computing Machinery, 22 Nov. 2021, dl.acm.org/doi/10.1007/978-3-030-89579-2_2.
5. Latner, Michael, et al. "Measuring Legislative Behavior: An Exploration of Digitaldemocracy.Org." *California Journal of Politics and Policy*, vol. 9, no. 3, 2017, pp. 1-11. *ProQuest*, <https://calpoly.idm.oclc.org/login?url=https://www.proquest.com/scholarly-journals/measuring-legislative-behavior-exploration/docview/1989192664/se-2>.
6. Nishal, Sachita, and Nicholas Diakopoulos. "From Crowd Ratings to Predictive Models of Newsworthiness to Support Science Journalism." *ACM Digital Library*, Association for Computing Machinery, 11 Nov. 2022, dl.acm.org/doi/abs/10.1145/3555542.
7. "NLTK." *Natural Language Toolkit*, NLTK Project, www.nltk.org/. Accessed 31 May 2025.
8. *Pandas*, pandas, pandas.pydata.org/. Accessed 31 May 2025.
9. Rybinski, Krzysztof. "Content Still Matters. A Machine Learning Model for Predicting News Longevity from Textual and Context Features." *Science Direct*, Elsevier Ltd., 25 Apr. 2023, www.sciencedirect.com/science/article/pii/S0306457323001358.
10. "Scikit-Learn." *Scikit-Learn*, scikit-learn.org/stable/. Accessed 31 May 2025.
11. Trilling, Damian, et al. "From Newsworthiness to Shareworthiness: How to Predict News Sharing Based on Article Characteristics." *Sage Journals*, Association for Education in Journalism & Mass Communication, 20 June 2016, www.researchgate.net/publication/328039672_httpjournalssagepubcomdoiabs1011770887302X07303626.
12. Xiong, Jie, et al. "DNCP: An Attention-Based Deep Learning Approach Enhanced with Attractiveness and Timeliness of News for Online News Click Prediction." *Science Direct*, Elsevier B.V., 11 Jan. 2021, www.sciencedirect.com/science/article/pii/S0378720621000021.
13. Zhang, Ye, et al. "Characterizing the (Perceived) Newsworthiness of Health Science Articles: A Data-Driven Approach." *JMIR Medical Informatics*, JMIR Publications Inc., July 2016, medinform.jmir.org/2016/3/e27.

9 Appendices

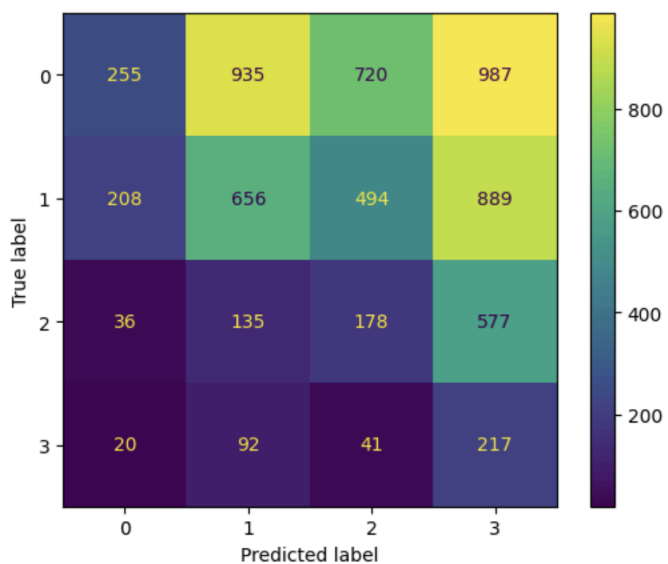
The output and confusion matrix below is the result of running random forests on the unprocessed transcript words. This was the most accurate model we ran.

Cross-validation accuracy: 0.8208770356547902



The model below was the result of running Scikit-Learn Multinomial Naive Bayes on the unprocessed transcript words with subsampling. While the samples are more balanced, the accuracy is significantly lower than the results with no subsampling.

Cross-validation accuracy: 0.4230769230769231



The following output shows some of the misclassified samples from running NLTK Naive Bayes on the unprocessed transcript words.

Average Accuracy: 0.36726241888689565

Some misclassified samples:

```

Some misclassified samples:
Features: {'Okay': 'high', '': 'high', 'number': 'medium', 'two': 'medium', '.': 'high', 'So': 'medium', 'chair': 'medium', 'Members': 'high'}
True Label: 0, Predicted Label: 15+

```

```
Features: {'And': 'medium', 'I': 'high', 'believe': 'medium', 'we': 'high', '"re": 'high', 'going': 'medium', 'to': 'high', 'be': 'high', 'p
True Label: 0, Predicted Label: 15+
```

```
Features: {'Mr': 'medium', '.': 'high', 'Chair': 'medium', 'and': 'high', 'Members': 'high', ',': 'high', 'thank': 'high', 'you': 'high', 'I': 'high'}
True Label: 0, Predicted Label: 15+
```

```
Features: {'Thank': 'high', 'you': 'high', '.': 'high', 'Next': 'high', ',': 'high', 'we': 'high', 'll': 'medium', 'move': 'high', 'to': 'high'}
True Label: 0, Predicted Label: 15+
```

```
Features: {'Clerk': 'high', 'will': 'medium', 'read': 'high', 'Assembly': 'high', 'Bill': 'high', '1227': 'medium', 'by': 'low', 'Member': 'True Label: 0, Predicted Label: 1-5
```

```
Features: {'Good': 'high', 'morning': 'high', ',': 'high', 'Chair': 'medium', 'Committee': 'high', 'Members': 'high', 'and': 'high', 'of': 'high'}
True Label: 0, Predicted Label: 15+
```

```
Features: {'Thank': 'medium', 'you': 'medium', ',': 'medium', 'Mr.': 'high', 'Speaker': 'high', '.': 'low', 'AB': 'medium', '249': 'high', 'True Label: 0, Predicted Label: 15+
```

```
Features: {'Thank': 'medium', 'you': 'medium', ',': 'medium', 'Mr.': 'high', 'Speaker': 'high', 'and': 'medium', 'colleagues': 'high', '.': 'medium'}
True Label: 0, Predicted Label: 6-14
```

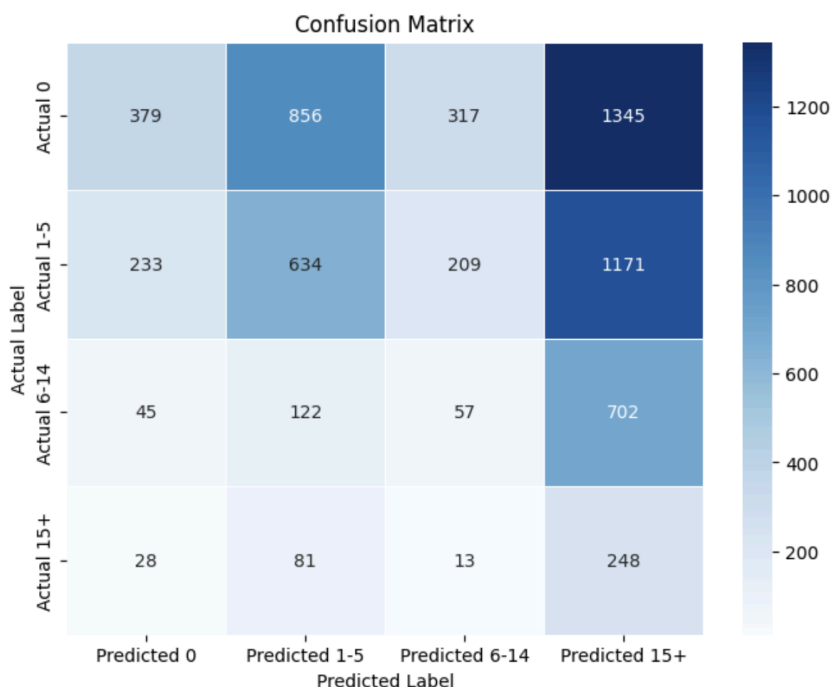
```
Features: {'7-0': 'high', '.': 'high', 'That': 'high', 'bill': 'medium', '"S": 'high', 'on': 'medium', 'call': 'high', 'I': 'medium', 'see':  
True Label: 0, Predicted Label: 6-14
```

```

Features: {'Okay': 'high', ',': 'high', 'now': 'high', 'we': 'high', 'will': 'high', 'take': 'high', 'up': 'high', 'Assembly': 'medium', 'B':
True Label: 0. Predicted Label: 15+

```

The model below shows the confusion matrix from running NLTK Naive Bayes on the unprocessed transcript words.



Below are the full accuracy score results of running each of the models.

NLTK Naive Bayes:

1. Unprocessed transcript words: 36.73%
2. Processed/stemmed transcript words: 30.96%
3. Processed bill analysis: 35.57%
4. Unprocessed bill analysis: 35.38%

Scikit-Learn Multinomial Naive Bayes:

1. Unprocessed transcript words: 78.36%
2. Unprocessed transcript words with subsampling: 42.31%
3. Processed transcript words: 76.74%
4. Processed bill analysis: 76.95%
5. Unprocessed bill analysis: 77.19%

Random Forest:

1. Unprocessed transcript words, before tuning: 81.95%
2. Unprocessed transcript words, after tuning: 82.09%
3. Processed transcript words: 81.77%
4. Processed bill analysis: 79.56%
5. Unprocessed bill analysis: 79.84%

Gradient Boosting:

6. Unprocessed transcript words: 81.99%
7. Processed transcript words: 81.09%
8. Processed bill analysis: 79.63%
9. Unprocessed bill analysis: 79.52%

Support Vector Machine:

1. Unprocessed transcript words: 82.03%
2. Processed transcript words: 82.07%
3. Processed bill analysis: 79.41%
4. Unprocessed bill analysis: 79.48%